

委员会报告

OpenCollab + GLM-5.2 的 SWE 委员会原理与实验

实验整理

2026-06-30

摘要

这份报告把 OpenCollab 中“委员会模式”的设计动机、 workflows 结构和 SWE-bench Lite 前 100 题上的补测结果单独整理出来。委员会模式的核心思想是把软件修复任务拆成定位、证据、验证、修复、补丁攻击和最终放行几个岗位，让模型在写生产补丁前先建立可审计证据，在写完补丁后再接受边界、回归和可见行为变化审查。报告特别展开这些岗位的具体输入、动作、产物和判断标准，避免把“契约”“验证”“攻击”停留在抽象名词上。

当前已经进入实验统计的 SWE 委员会补测使用 V1 工作流，代码名为 `validation-council-solve`。

1 问题背景

SWE-bench 的任务表面上是“给定一个 issue，让模型改代码”，实际难点在于隐藏测试检查的是更精确的行为契约。这里的“行为契约”指可被程序观察到的要求，例如某个函数应返回什么值、应抛出哪类异常、是否应发出 warning、输出顺序是否稳定、迁移前后数据库里应该留下什么记录、某个选项是否应该被忽略。模型在生成补丁时只能看到问题文本、起始版本仓库、公开测试和公开文档。官方隐藏测试、官方评分补丁、FAIL_TO_PASS 测试节点在生成阶段都不可见，只能在最终评分阶段使用。

这里有一个直接的矛盾。修复者如果过早写代码，很容易把某个合理猜测写成生产行为。测试者如果缺少证据分层，也可能把没有来源的猜测写成临时测试预期。委员会模式尝试把这两个风险拆开处理。它要求不同角色先回答“证据支持什么”，再回答“代码应该怎么改”，最后回答“补丁有没有改出新的可见行为风险”。

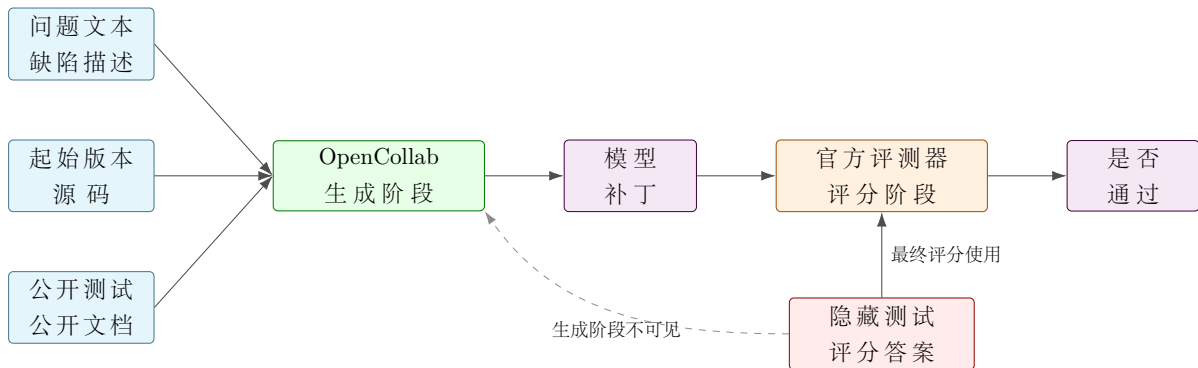


图 1: SWE-bench 盲验证口径下，模型生成阶段和官方评分阶段的边界

图里最关键的一条边界，是委员会可以构造自己的临时验证，但这些验证必须来自公开信息。所谓“验证更强”，指的是模型和工作流从公开信息里挖出更多行为约束，而不是提前看到官方隐藏测试。

2 从团队到委员会

OpenCollab 原有团队模式依赖负责人根据现场情况调度角色。负责人可以安排测试员、修复者和评审者，也可以根据中间结果决定继续探索或停止。这样的方式灵活，但重复实验时控制流会随负责人判断变化，某些关键步骤也可能被跳过。

工作流模式把控制权放进 Python 代码。节点顺序、并行扇出、最大重试次数、放行条件和失败分支都由代码固定，每个节点内部仍由模型完成推理和文本生成。委员会模式就是这种思路在 SWE-bench 上的专门版本。它保留多角色协作，但把岗位关系和状态流转固定成可审计的实验协议。

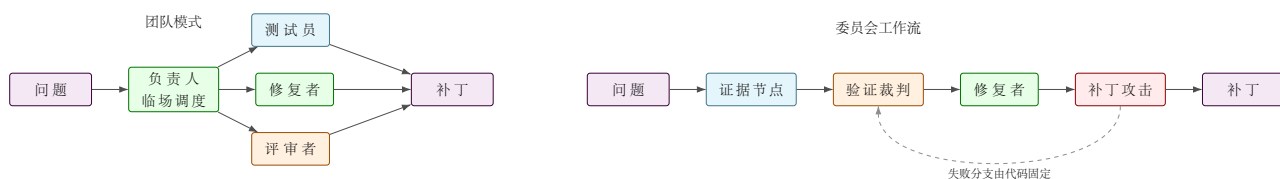


图 2: 团队模式与委员会工作流的控制权差异

这种结构带来的科研价值在于可复查。若某个补丁失败，报告可以追问失败来自契约证据不足、边界验证不足、回归审查不足，还是修复者实现错误。若某个补丁成功，也可以观察成功来自更准的定位、更强的验证，还是补丁攻击阶段拦下了错误方向。

3 术语拆解：这些岗位具体做什么

“契约提取”听起来抽象，实际动作很具体。契约挖掘者会先读 issue 原文、issue 讨论中公开的复现描述、相关源码、公开测试和公开文档，然后把它们改写成若干条可观察行为。每条行为都要说明触发条件、观察对象、期望结果和证据来源。例如“当自引用对称 ManyToManyField 显式传入 `related_name` 时，应出现 `fields.W345 warning`”就是一条契约。反过来，“所有 ManyToManyField 都要出现这个 `warning`”就缺少证据，应该被降级为猜测或直接拒绝。

“测试地图”负责把一个行为要求翻译成仓库内可执行的验证位置。测试地图员要回答的是这个仓库里到底怎样验证一个行为，包括测试命令是什么、已有测试放在哪些目录、测试夹具如何初始化、公开测试里有没有相近断言、临时 probe 应该放在哪里、运行后怎样清理、哪些文件不能进入最终 diff。对 Django 任务来说，它通常会定位到 `tests/runtests.py`、某个 `tests/...` 目录、model check 的断言风格，以及可用的最小 Python probe。

“验证”分成生成和裁判两步。验证生成者把契约改成候选检查，例如一段临时 Python 脚本、一个公开测试命令、一个 grep 文档条目的命令。验证裁判只保留证据足够的检查。证据强的检查可以写成硬预期，例如“这三种输入分别应产生 warning、无 warning、无 warning”。证据弱的检查只能做诊断，例如“这个边界值得观察，但不能作为失败标准”。这样可以减少模型把猜测写成测试答案的风险。

“补丁攻击”发生在修复者已经改完代码之后，审查对象是当前 diff 背后的行为假设。分支攻击会

改变输入组合，看补丁是否只覆盖了最显眼的分支。回归扫描会找相邻公开行为，看补丁有没有破坏旧功能。可见行为审查会检查返回值、异常、warning 文本、默认值、顺序和文档条目是否被证据支持。攻击阶段最后产出一组需要修复者处理的具体风险。

术语	具体动作	合格产物长什么样
定位	找到最可能相关的文件、函数、类和调用路径，写出根因假设。	指向可读源码位置，并说明为什么这些位置和 issue 症状有关。
契约提取	把公开材料改写成可观察行为，标注触发条件、观察对象、期望结果和证据来源。	每条契约都能追到 issue、源码、公开测试或文档，缺证据的内容被标为猜测。
测试地图	找测试框架、测试命令、夹具、相邻测试、临时 probe 写法和清理规则。	给出可运行命令和最小验证方式，同时说明临时文件不进入最终 diff。
修复前验证	在起始版本上运行或规划检查，区分 base-fail、base-pass、invalid、weak。	修复者知道哪些检查复现 bug，哪些只是诊断，哪些检查无效。
修复	在生产源码中做最小改动，避免改测试或提交临时文件。	diff 只包含与 issue 根因相关的源码或文档改动。
补丁攻击	围绕当前 diff 设计边界输入、相邻行为检查和可见行为审查。	输出具体风险，例如“这个 warning 是否只应在自引用字段触发”。
最终验证	读取最终 diff、运行可行测试、确认临时文件缺席、列出允许提交的路径。	给出 PASS/FAIL、允许路径、拒绝路径和失败理由。

表 1: 委员会术语的可执行含义

4 委员会 V1：实际运行的验证委员会

当前已经跑过并进入统计的委员会实验使用 V1，代码名是 validation-council-solve。所有 SWE 委员会补测脚本传入的工作流参数都是 --workflow validation-council-solve，并开启盲验证。V1 的设计目标是把“写代码”和“验证代码”拆开。修复者写补丁前要先接收契约和测试地图，写完补丁后要经过补丁验证、风险审计、修复后验证和最终验证。

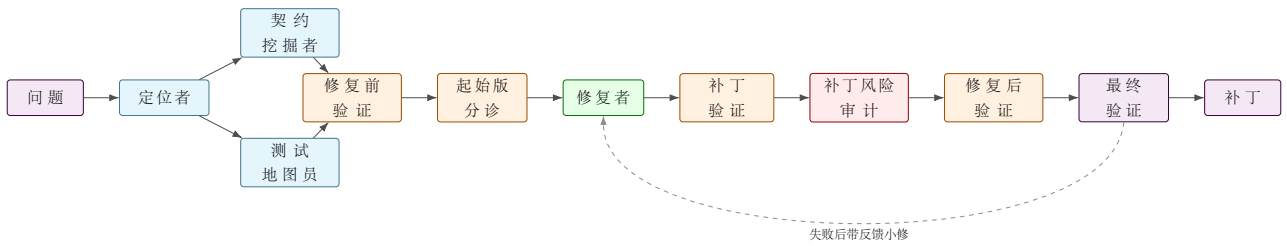


图 3: 委员会 V1：验证委员会的实际运行结构

V1 的前半段负责建立证据。定位者先把 issue 的自然语言症状变成源码范围，例如“相关逻辑可能在某个字段检查函数、迁移函数或渲染函数附近”。契约挖掘者接着把问题拆成期望行为、当前错误行为和不应改变的旧行为。测试地图员补充仓库层面的验证信息，说明应该用哪个测试命令、哪些公开测试最接近、临时 probe 怎样写才不会污染最终 diff。随后，修复前验证工厂把这些材料改成候选检查，修复前验证裁判最多保留 5 个有证据支撑的候选，起始版分诊器再判断这些候选在原始版本上的状态。

V1 的后半段负责审查补丁。修复者最多进行 2 轮尝试，每轮之后由补丁验证者检查差异边界，补丁风险审计者寻找边界和回归风险，修复后验证工厂和裁判生成并筛选新的检查，最终验证者读取差异、运行可行的公开测试和已批准验证，并给出允许进入模型补丁的路径。最终验证者通过时，工作流产出 guarded staged diff。最终验证者失败时，失败摘要会反馈给下一轮修复者。

换成一次真实运行来看，V1 的每一步都对应一个可检查的中间产物。以 `django__django-14730` 为例，这个 issue 说的是自引用对称 `ManyToManyField` 上 `related_name` 实际无效，但 Django 当时没有给出任何提示。定位者把范围缩到 `django/db/models/fields/related.py` 的 `ManyToManyField` 检查逻辑，以及 `docs/ref/checks.txt` 的检查项文档。契约挖掘者从中抽出三条核心行为。显式传入 `related_name` 且关系是自引用对称时应给 warning，没有传入 `related_name` 时不应给这个 warning，已有的 W340、W343 等检查则要保持原样。测试地图员接着指出可用的验证方式，包括构造最小 Django model、调用字段检查函数、运行相关 model check 公开测试，再检查文档里是否补充了 W345。

后面的角色继续把这些产物接上。修复前验证裁判会接受能区分三种输入的临时 probe，也会接受公开测试命令和文档 grep。修复者据此保存用户原始传入的 `related_name`，在检查函数里新增 `fields.W345` warning，并补上文档条目。补丁风险审计者再看这个 diff 是否把 warning 范围放得过宽，是否误伤非自引用字段，是否把 warning 变成 error，是否漏了文档。最终验证者只允许 `django/db/models/fields/related.py` 和 `docs/ref/checks.txt` 进入预测补丁，并确认临时 probe 没有留在 diff 里。这个例子说明，“委员会”真正做的是把一条 issue 逐步转成一组可观察行为、一组验证命令和一个受约束的生产 diff。

角色	主要输入	具体动作	主要产物
定位者	问题文本、源码、公开测试	找相关文件和根因路径，写完成条件。	根因假设、相关文件、完成条件
契约挖掘者	定位结果、公开信息	把自然语言症状改写成可观察行为。	行为契约、无影响行为、证据来源
测试地图员	仓库测试结构	找测试命令、夹具、相邻测试和临时 probe 写法。	可运行命令、测试入口、临时验证建议
修复前验证裁判	候选验证、行为契约	接受强证据检查，拒绝猜测预期，保留诊断信号。	被批准的临时检查、拒绝理由
修复者	契约、测试地图、分诊结果、失败反馈	只改生产源码或必要文档，保持 diff 最小。	最小生产补丁
补丁风险审计者	当前差异、契约、测试地图	找补丁覆盖不到的分支、边界和相邻回归。	边界风险、回归风险、需补充验证的字段
最终验证者	当前差异、所有验证结果	查 diff、跑可行测试、确认临时文件缺席。	放行判定、允许提交路径

5 委员会 V2：后续图同构设计

V2 的代码名是 `swe-committee-v2`。它来自 V1 的经验，但结构更细，目标是让证据阶段、仲裁阶段、补丁攻击阶段和最终放行阶段严格分工。V2 增加了可见行为清单员和契约仲裁庭。可见行为清单员会把返回值、异常类型、warning 文本、错误码、默认值、排序、数据库副作用、文档承诺等外部可观察字段列出来。契约仲裁庭再把证据分成强证据、弱证据、猜测字段和禁止硬断言字段。强证据可以生成硬测试，弱证据只能生成诊断，猜测字段只保留为假设，禁止字段会被后续最终怀疑者重点检查。

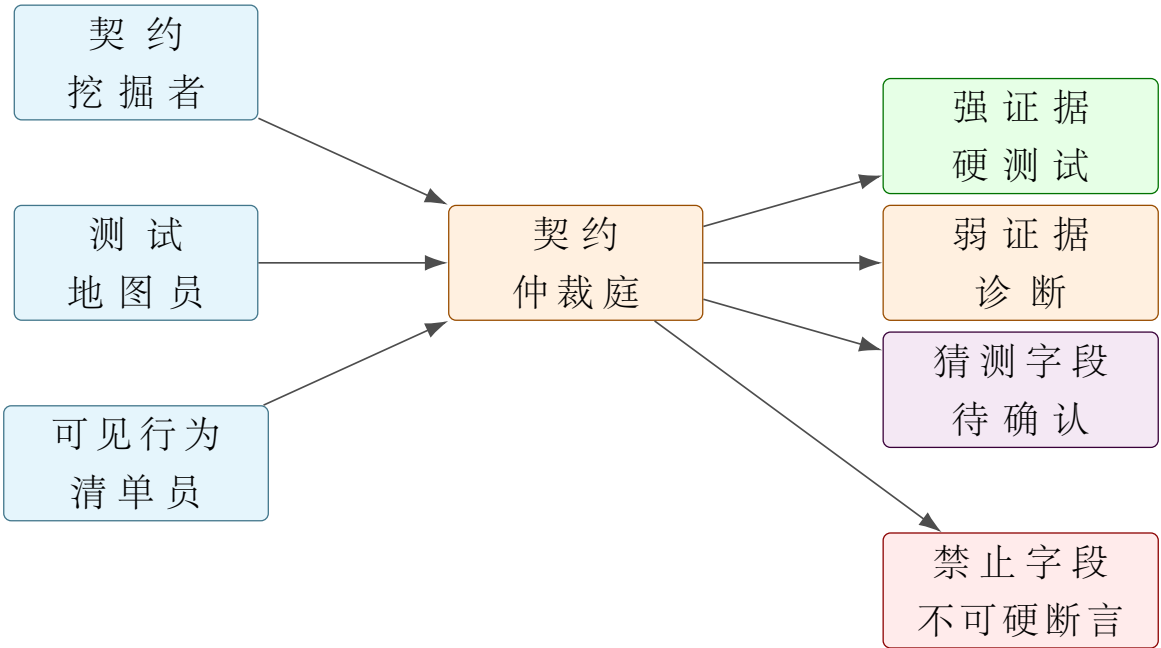


图 4: 委员会 V2 的证据分层

V2 的补丁后阶段也比 V1 更明确。已有测试和批准验证先检查补丁状态，随后分支攻击者寻找条件分支和边界输入，回归扫描者关注相邻行为是否被破坏，可见变化审查者检查返回值、异常、警告、文本、默认值和顺序等外部可见字段是否被证据授权。修复后验证裁判看到有证据失败时，直接把任务交给修复者小修。最终怀疑者看到契约证据不足时，任务回到契约仲裁庭。最终验证者看到具体提交问题时，任务回到修复者。

V2 里“强证据”和“弱证据”的差别也可以具体说明。强证据通常来自 issue 明确描述、公开测试已有断言、源码注释或文档承诺，可以写成必须满足的检查。弱证据通常来自相邻实现、合理推断或风险审计，它可以提醒模型观察某个方向，但不能直接写成 expected output。猜测字段表示当前材料无法支持，最终怀疑者会阻止补丁依赖这些字段。比如某个 issue 只说“应该发出 warning”，但没有说明 warning 文本必须逐字相同，那么“有 warning”可能是强证据，“具体英文句子逐字匹配”就需要额外文档或源码证据。

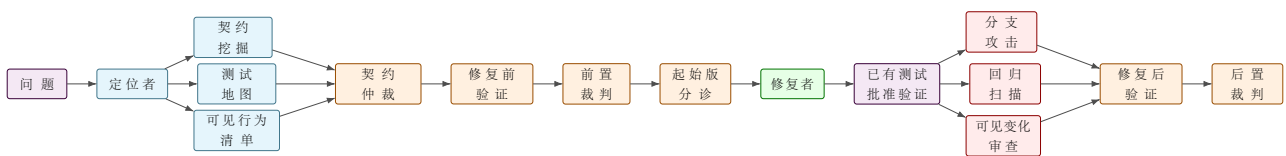


图 5: 委员会 V2 的主结构：三路证据、前置验证、三路补丁攻击

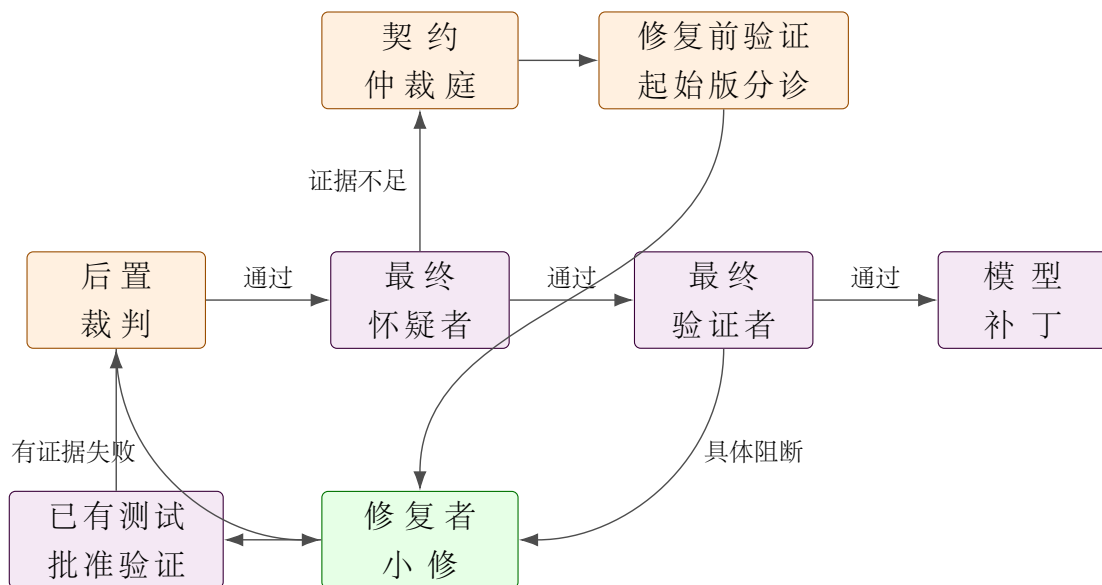


图 6: 委员会 V2 的后半段状态机

V1 和 V2 的关系并不复杂。V1 把验证从修复者手中独立出来，V2 再把“验证失败”细分为有证据失败、证据不足和具体提交阻断。V1 更轻，适合快速看清验证 workflow 本身能带来多少收益。V2 更重，适合研究更强的 validation data 能不能减少那些只差一两个 corner case 的失败。

维度	委员会 V1	委员会 V2
前置证据	契约挖掘者和测试地图员并行产出证据。	契约挖掘者、测试地图员、可见行为清 单员三路并行。
证据处理	契约集合直接供验证生成者和修复 者使用。	契约仲裁庭将证据分成强证据、弱证 据、猜测字段、禁止字段。
补丁后审查	补丁验证、风险审计、修复后验证、 最终验证线性串联。	分支攻击、回归扫描、可见变化审查并 行进入后置裁判。
失败分流	最终验证失败后带反馈重试，最多 2 轮修复。	有证据失败回小修，证据不足回仲裁， 具体阻断回小修。
实验状态	已用于剩余 13 题补测，并进入 89/100 统计。	已实现为可调用工作流，后续实验需单 独统计。

V2 状态	触发条件	工作流下一步
有证据失败	已批准检查在补丁后仍失败，且 检查的预期来自强证据。	交给修复者小修，反馈具体失败输入和 观察结果。
证据不足	补丁改变了可见行为，但仲裁庭 没有给出足够证据。	回到契约仲裁庭，重新判定该字段能否 写成硬预期。
具体阻断	diff 中有临时文件、允许路径不干 净、公开测试未通过。	交给修复者清理或最小修补。
最终通过	强证据检查通过，诊断信号无阻 断，diff 只含允许路径。	产出模型补丁，交给官方评测器。

6 实验口径

委员会补测发生在前 100 题已经达到 87/100 之后。此时剩余 13 个样本仍未解决，因而成为委员会 V1 的补测对象。生成阶段使用 GLM-5.2，并通过 `swebench/gen_prediction_workflow.py` 调用 `validation-council-solve`。该驱动对 `validation-council-solve` 和 `swe-committee-v2` 默认启用盲验证保护，只向工作流提供公开信息，并用 `guarded staged diff` 作为最终预测补丁。

主批次预算从 1M token 开始，后续根据预算信号升到 2M 和 4M，针对性补跑则用更高预算处理仍需复核的样本。官方结果只采用 SWE-bench harness 生成的评测报告。如果某个样本没有形成干净的非空 patch，实验表里记为未评测，保守累计分数把未评测样本一律算作尚未解决。

7 总体结果

委员会 V1 在剩余 13 个样本上形成了 10 个可评测非空 patch，其中 2 个通过、8 个未通过，另外 3 个样本没有干净非空 patch 进入正式统计。新增通过样本是 `django__django-11019` 和 `django__django-14730`。这样一来，前 100 题保守累计结果从 87/100 提升到 89/100。如果只看已有官方报告的 97 个样本，通过数为 89/97。

口径	已通过	未通过	未评测
委员会补测的 13 个剩余样本	2	8	3
前 100 题累计，未评测按未解决计	89	11	0
前 100 题累计，仅看已有官方报告样本	89	8	3

表 2: 委员会 V1 补测后的总体结果

实例	状态	apply	F2P 失败	P2P 失败	patch 字符	首个失败信号或备注
astropy__astropy-14365	未通过	是	1	0	475	test_roundtrip[True]
astropy__astropy-7746	未通过	是	1	0	995	test_zero_size_input
django__django-11019	通过	是	0	0	5063	新增通过样本
django__django-11283	未评测	—	—	—	0	没有干净非空 patch
django__django-11564	未通过	是	2	0	948	SCRIPT_NAME 下 static/media 前缀行为仍不完整
django__django-11630	未通过	是	2	0	1671	duplicate db table checks 仍有残留失败
django__django-11905	未通过	是	2	0	1119	isnull 非布尔值与 iterator 行为仍未覆盖
django__django-13321	未通过	是	18	0	787	session decode 相关失败较重
django__django-13768	未通过	是	1	0	911	send_robust logging 行为仍未满足
django__django-14155	未通过	是	3	0	1792	ResolverMatch repr 对 partial 的处理仍不足
django__django-14730	通过	是	0	0	1699	新增通过样本
django__django-15252	未评测	—	—	—	0	干净委员会补测未形成正式非空 patch
django__django-15695	未评测	—	—	—	0	没有干净非空 patch

表 3: 委员会 V1 补测逐题结果

8 新增通过样本

django__django-11019 是 Django forms media 合并问题。委员会补测最终生成的 patch 修改 django/forms/widgets.py，把两个列表合并逻辑扩展为多列表合并，并使用稳定拓扑排序保留依赖顺序。在这道题里，“契约”可以拆成两条具体要求。一是多个 media 列表合并后依赖顺序要尽量保留，二是当两个文件的相对顺序冲突时系统仍要发出冲突 warning。测试地图则会关注 Django 里 media 渲染和 widget 相关公开测试怎么跑。这个样本说明，委员会的价值在于修复前契约和补丁后验证一起约束了排序语义，而单纯做局部拼接，很容易遗漏冲突 warning 或多列表排序。

django__django-14730 是自引用对称 ManyToManyField 上 related_name 无效但缺少警告的问题。委员会补测生成的 patch 修改 django/db/models/fields/related.py 和 docs/ref/checks.txt，新增 fields.W345 警告，并记录用户传入的 related_name，防止后续 contribute_to_class()

自动改写字段后丢失原始意图。这个样本里的强证据包括“自引用对称关系中 `related_name` 没有效果”和“Django check framework 应以 warning 形式提示无效选项”。弱证据则是一些人工构造的边界 probe，例如未绑定 field 的中间状态。委员会阶段明确区分运行时检查、文档条目、已有 warning 的回归保护和自引用边界，最终官方 F2P 与 P2P 都为 0。

这两个通过样本有一个共同点。它们要解决的都不是“缺一行代码”式的问题，而是要把行为要求、相邻测试、文档和 warning 体系一起对齐。委员会结构在这类样本上更容易发挥作用，因为它把“该检查什么”提前到了修复者写代码之前。

9 失败与未评测样本

已评测但未通过的 8 个样本有一个共同特征。它们的 patch 都能 apply，P2P 回归都是 0，失败全部来自 F2P 残留。这说明委员会 V1 没有把问题卡在 patch 格式或回归破坏上，真正的缺口是语义修复没有完全覆盖隐藏测试行为。

`django__django-13321` 的残留失败最重，F2P 失败达到 18 个，说明补丁方向与隐藏测试契约距离较大。`django__django-14155` 残留 3 个失败，`django__django-11564`、`django__django-11630` 和 `django__django-11905` 各残留 2 个失败。这类样本更像“已经修到一部分，但少了关键边界”。`astropy__astropy-14365`、`astropy__astropy-7746` 和 `django__django-13768` 各残留 1 个失败，它们最适合留给 V2 的补丁攻击阶段继续研究，看分支攻击、回归扫描和可见行为审查能不能找出最后那个 corner case。

未评测的 3 个样本按保守口径计为未解决。`django__django-11283` 和 `django__django-15695` 没有形成干净非空 patch。`django__django-15252` 的干净委员会补测没有形成正式非空 patch。它先前存在过一个辅助候选并完成过官方评测，但那属于辅助材料，最终统计仍按干净委员会补测口径处理。

10 对验证数据的启发

委员会实验指向一个明确方向。剩余失败大多不是补丁无法应用，也很少是 P2P 回归，真正缺的是更强的验证数据。这里的“更强”必须守住盲验证边界，既不去碰隐藏测试，也不把失败题的答案预先写给模型。可行的做法是让工作流在看到每一道题之后，自己从公开材料里抽取契约、相邻公开测试、可见行为字段和风险边界，再把这些材料交给裁判筛选。

V1 已经证明结构化验证能带来增量，13 个剩余样本里通过了 2 个。V2 则把下一步的研究问题写得更清楚。在没有官方隐藏测试信息的前提下，增加可见行为清单、契约仲裁、分支攻击、回归扫描和最终怀疑者，能不能把“只差一个边界”的样本转化为通过样本。如果后续继续实验，最值得观察的指标包括强证据临时检查的数量、弱证据诊断的数量、被最终怀疑者拦下的证据不足字段，以及每个未通过样本的首个 F2P 残留能否被 V2 提前预警。

11 可复查文件

文件或目录	用途
workflows/validation_council_solve.py	委员会 V1 的实际实现，包含定位、证据、修复前验证、修复、补丁风险审计和最终验证。
workflows/swe_committee_v2.py	委员会 V2 的图同构实现，包含可见行为清单、契约仲裁庭、补丁攻击阶段和最终怀疑者。
swebench/gen_prediction_workflow.py	SWE-bench 工作流驱动，负责盲验证保护、预算、容器内生成和 guarded staged diff 抽取。
eval_work/swe_committee_mode_remaini ng13_glm52_20260629	委员会 V1 剩余 13 题主批次生成、metrics、predictions 和分批官方评测。
eval_work/swe_committee_targeted_12_ 34_8m_glm52_20260630	针对性补跑和官方评测材料，包含 django__django-11019 的通过结果。
committee_mode_results.tex	较早的委员会结果小结，保留逐题结果和截止口径，位于本报告同一目录。
committee_report.tex	当前独立委员会报告源码，位于本报告同一目录。

表 4: 委员会报告的主要可复查材料

12 结论

委员会模式把 SWE-bench 解题从“让模型直接修”推进到“先建立证据，再写补丁，再攻击补丁，最后放行”的可审计结构。V1 已经在剩余 13 个难样本上新增解决 2 个，使前 100 题保守累计结果达到 89/100。失败样本的共同形态是 patch 可应用、P2P 为 0、F2P 仍有残留，说明后续提升重点应放在更强的盲验证和边界探索上。

V2 已经把这一方向拆成更细的组织结构，包含三路证据、契约仲裁、前置验证、三路补丁攻击、后置裁判、最终怀疑者和最终验证者。下一轮实验可以把 V2 单独作为新条件，与 V1 和 Team 模式分开统计，重点看它能不能减少“只差一两个 corner case”的语义失败。